PRESIDENCY UNIVERSITY
Private University Estd. in Karnataka State by Act No. 41 of 2013
Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

Real-Time Object Detection System Using YOLOv5

AN INTERNSHIP REPORT

Submitted by

VARSHA D - 20221COM0200

Under the guidance of,

Mr. IRFAN RAJAB BHAT

Assistant Professor

BACHELOR OF TECHNOLOGY
IN
COMPUTER ENGINEERING
(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

PRESIDENCY UNIVERSITY
BENGALURU
APRIL 2026

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

Certified that this report **"Real-Time Object Detection System Using YOLOv5"** is a Bonafide work of **VARSHA D (20221COM0200)**, who has successfully carried out the internship work and submitted the report for partial fulfilment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in COMPUTER ENGINEERING (Artificial Intelligence and Machine Learning) during 2025-2026.

Mr. IRFAN RAJAB BHAT
Assistant Professor
PSCS
Presidency University

Mr. Shreehari T.M
Program Internship Coordinator
PSCS
Presidency University

Dr. Md Ziaur Rahaman
School Internship Coordinator
PSCS
Presidency University

Dr. Gopal Krishna Shyam
Head of the Department, PSCS
Presidency University

Dr. Duraipandian N
Dean, PSCS / PSIS
Presidency University

Name and Signature of the Examiners

Sl. No.	Name	Signature	Date
1			
2			

PRESIDENCY UNIVERSITY

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

I am a student of final year B.Tech in COMPUTER ENGINEERING (Artificial Intelligence and Machine Learning), at Presidency University, Bengaluru, named VARSHA D, hereby declare that the internship work titled "Real-Time Object Detection System Using YOLOv5" has been independently carried out by me and submitted in partial fulfillment for the award of the degree of B.Tech in COMPUTER ENGINEERING (Artificial Intelligence and Machine Learning) during the academic year of 2025-2026. Further, the matter embodied in the internship has not been submitted previously by anybody for the award of any Degree or Diploma to any other institution.

VARSHA D 20221COM0200

Signature: _____

PLACE: BENGALURU
DATE: 24 April 2026

INTERNSHIP COMPLETION CERTIFICATE

Date: 24-12-2025

From,

VARSHA D
20221COM0200
Presidency University, Bangalore

To,

The Dean,

Presidency School of CSE,

Presidency University, Bangalore

Subject: Internship Offer Letter – Visual Analytics | January 2026

Respected Sir/Madam,

I, Ms. Varsha D, Roll No. 20221COM0200, Student of Computer Engineering (Artificial Intelligence and Machine Learning) program studying in 8th semester received an internship offer letter from BLP Industry.AI Private Limited.

Domain: Visual Analytics

Duration: Five (5) months

During the internship period, Ms. Varsha D will gain hands-on exposure and practical experience in the field of Visual Analytics. Upon successful completion of the internship, she will be provided with an Internship Completion Letter.

Bangalore Office Address:
BLP Industry.AI, 11th Floor, Gamma Block, Sigma Soft Tech Park 7,
Whitefield Main Road, Varthur Kodi, Bangalore – 560066, Karnataka.

Yours Truly,
Name: Varsha D
Mobile No: 9741492145

ACKNOWLEDGEMENTS

For completing this internship work, I have received the support and the guidance from many people whom I would like to mention with deep sense of gratitude and indebtedness. I extend my gratitude to our beloved Chancellor, Pro-Vice Chancellor, and Registrar for their support and encouragement in completion of the internship.

I would like to sincerely thank my industry guide AI Engineer, BLP Industry.AI Private Limited, for their constant mentorship, patient guidance, and technical expertise throughout the internship. Their insights into the practical nuances of Computer Vision and AI systems development greatly enriched my learning experience.

I would like to sincerely thank my internal guide Mr. IRFAN RAJAB BHAT, Assistant Professor, Presidency School of Computer Engineering, Presidency University, for his moral support, motivation, timely guidance and encouragement provided to me during the period of internship work.

I am also thankful to Dr. Gopal Krishna Shyam, Professor, Head of the Department, Presidency School of Computer Science and Engineering, Presidency University, for his mentorship and encouragement.

I express my cordial thanks to Dr. Duraipandian N, Dean PSCS & PSIS and the Management of Presidency University for providing the required facilities and intellectually stimulating environment that aided in the completion of my internship work.

I am grateful to Dr. Md Ziaur Rahaman, Internship Coordinator and Mr. Shreehari T M, Program Internship Coordinator, Presidency School of Computer Science and Engineering, for facilitating problem statements, coordinating reviews, monitoring progress, and providing their valuable support and guidance.

I am also grateful to Teaching and Non-Teaching staff of Presidency School of Computer Science and Engineering and also staff from other departments who have extended their valuable help and cooperation.

VARSHA D

ABSTRACT

Object detection—the automated identification and localization of objects within images or video streams—is a core capability in modern Artificial Intelligence (AI) and Computer Vision (CV). This report summarizes an internship conducted at BLP Industry.AI Private Limited, Bengaluru, from January 2026, during which a production-oriented real-time object detection system was designed, developed, and deployed using the YOLOv5 (You Only Look Once, version 5) framework. The project covered the complete AI development lifecycle, including data collection from COCO datasets and custom recordings, image annotation using LabelImg in YOLO format, dataset preprocessing and YAML configuration, and model training using transfer learning on GPU hardware with CUDA acceleration.

The system was rigorously evaluated using performance metrics such as Precision, Recall, and Mean Average Precision (mAP), and integrated into a real-time inference pipeline built with Python and OpenCV for live video processing, incorporating Region-of-Interest (ROI)-based violation detection and alert generation. The final system achieved reliable real-time performance, demonstrating that a well-structured pipeline combining quality data preparation, transfer learning, and systematic evaluation can produce production-grade object detection solutions, supported by detailed architectural explanations, implementation code, and performance analysis.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
AP	Average Precision
CCTV	Closed-Circuit Television
CHW	Channel x Height x Width (tensor format)
CNN	Convolutional Neural Network
COCO	Common Objects in Context
CPU	Central Processing Unit
CUDA	Compute Unified Device Architecture
CV	Computer Vision
DNN	Deep Neural Network
FP	False Positive
FP16	16-bit Floating Point (half-precision)
FN	False Negative
GPU	Graphics Processing Unit
IP	Internet Protocol
mAP	Mean Average Precision
ML	Machine Learning
ONNX	Open Neural Network Exchange
OpenCV	Open Source Computer Vision Library
ReLU	Rectified Linear Unit
ROI	Region of Interest
RTSP	Real-Time Streaming Protocol
SGD	Stochastic Gradient Descent
TP	True Positive
TN	True Negative
YAML	YAML Ain't Markup Language
YOLO	You Only Look Once
YOLOv5	YOLO Version 5 (Ultralytics)

Chapter 1

INTRODUCTION

1.1 Background

The rapid growth of Artificial Intelligence (AI) and Computer Vision (CV) has opened transformative avenues for automating complex visual perception tasks traditionally performed by human operators. Object detection — the ability of a computer system to identify and precisely localize objects within images or video streams — forms a foundational component of modern AI applications, ranging from industrial safety monitoring and autonomous vehicles to smart surveillance, quality inspection, and medical imaging diagnostics.

Traditional image processing methods relied on hand-crafted features such as Histogram of Oriented Gradients (HOG), Scale-Invariant Feature Transform (SIFT), and Viola-Jones cascades — which required significant domain expertise and proved brittle against environmental variability. The emergence of Convolutional Neural Networks (CNNs) revolutionized the field by enabling the automatic learning of hierarchical feature representations directly from raw pixel data, outperforming traditional methods across virtually all detection benchmarks.

The YOLO (You Only Look Once) family of models represents one of the most significant milestones in real-time Computer Vision. Introduced by Redmon et al. (2016), YOLO reformulated detection as a unified regression problem — predicting bounding boxes and class probabilities from the full image in a single neural network forward pass. This single-pass approach eliminated the costly region proposal stages of two-stage detectors (e.g., Faster R-CNN), enabling a compelling combination of detection speed and accuracy ideal for production deployments.

1.2 Statistics of Internship

The internship was conducted at BLP Industry.AI Private Limited, Bengaluru, a technology company specializing in AI and Computer Vision solutions. Key statistics of the internship engagement are summarized below.

Parameter	Details
Organization	BLP Industry.AI Private Limited, Bengaluru
Internship Domain	Artificial Intelligence and Computer Vision (Visual Analytics)
Duration	January 2026 to July 2026 (5 months)
Primary Project	Real-Time Object Detection System Using YOLOv5
Technologies Used	Python, YOLOv5, OpenCV, PyTorch, CUDA, LabelImg, TensorBoard, Git
Industry Guide	AI Engineer, BLP Industry.AI Pvt. Ltd.
Infrastructure	GPU workstation (NVIDIA CUDA), Linux (Ubuntu 20.04 LTS)
Dataset Size	~1000-2000 annotated images (custom + COCO subsets)
Output Delivered	Deployed real-time object detection system with ROI-based violation alert logic

Table 1.0 — Internship Statistics Summary

1.3 Prior Existing Technologies

Before the development of the YOLO architecture and single-stage detectors, several generations of object detection approaches were dominant in both research and industry applications. Understanding these prior methods provides important context for appreciating the significance of the YOLOv5 approach adopted in this project.

Approach	Key Methods	Limitations
Hand-crafted Features	HOG + SVM, Viola-Jones, SIFT + Sliding Window	Brittle to lighting/scale changes; poor generalization; very slow at inference
Two-Stage CNN Detectors	R-CNN, Fast R-CNN, Faster R-CNN	High accuracy but slow; separate region proposal + classification stages; not real-time
Single-Stage Detectors (pre-YOLO)	SSD (Single Shot MultiBox Detector)	Faster than two-stage but lower accuracy, especially on small objects
YOLOv1-v4	YOLO, YOLOv2, YOLOv3, YOLOv4	Progressive improvements; YOLOv4 state-of-art but complex training pipeline
YOLOv5 (selected)	PyTorch-native, modular, scalable variants (nano to extra-large)	Best-in-class speed-accuracy balance; simple training API; production-ready

Table 1.1 — Prior Existing Technologies vs. Proposed YOLOv5 Approach

1.4 Proposed Approach — YOLOv5

The proposed approach for this internship project was the development of a complete, production-aligned real-time object detection system using the YOLOv5 framework. YOLOv5 (Ultralytics, 2020) was selected as the detection backbone based on the following justifications:

- Speed-Accuracy Balance: YOLOv5s achieves real-time inference speeds (60+ FPS on modern GPUs) while maintaining competitive mAP on standard benchmarks — making it ideal for live video processing.
- PyTorch Native Implementation: YOLOv5 is built on PyTorch, providing seamless CUDA GPU acceleration, dynamic computation graphs for debugging, and straightforward integration with the Python AI ecosystem.
- Transfer Learning Support: Pre-trained COCO weights enable rapid fine-tuning on custom datasets, dramatically reducing the labeled data and training time required.
- Scalable Model Variants: YOLOv5 offers five model sizes (n/s/m/l/x) allowing flexible speed-accuracy trade-offs for different deployment hardware targets.
- Production Deployment: Native export to ONNX, TensorRT, and CoreML formats enables seamless cross-platform deployment from development to edge/cloud production environments.
-

1.5 Objectives

The internship was structured around the following well-defined technical and professional objectives:

- Understand the theoretical foundations of AI and Computer Vision, including YOLO architecture, CNN-based feature extraction, anchor-based detection, and performance evaluation metrics.
- Develop proficiency in Python programming for building end-to-end AI applications including data processing pipelines, training scripts, and real-time inference systems.
- Build a complete end-to-end object detection system using YOLOv5 covering data collection, annotation, preprocessing, training, evaluation, real-time deployment, and ROI-based alert logic.
- Gain hands-on experience with GPU-accelerated computing using CUDA and PyTorch for deep learning model training and inference.
- Develop proficiency in image annotation using LabelImg and understand the critical importance of annotation quality in supervised learning systems.
- Learn and apply model optimization techniques including transfer learning, hyperparameter tuning, data augmentation, and early stopping.
- Develop professional skills including technical documentation, version control (Git), structured problem-solving, and team collaboration in an industry engineering environment.

1.6 Sustainable Development Goals (SDGs)

This internship project contributes to the following United Nations Sustainable Development Goals (SDGs):

- SDG 8 — Decent Work and Economic Growth: The internship developed industry-relevant AI engineering skills, directly enhancing the intern's employability and contributing to a skilled technical workforce in the growing AI industry.
- SDG 9 — Industry, Innovation and Infrastructure: The developed object detection system directly supports industrial automation, safety compliance monitoring, and intelligent infrastructure management — core applications of Industry 4.0.
- SDG 3 — Good Health and Well-being: Safety equipment compliance detection systems built on this technology can prevent workplace injuries and fatalities by ensuring workers wear required PPE.
- SDG 11 — Sustainable Cities and Communities: The system's applicability to smart city surveillance, public safety monitoring, and traffic management supports the development of safer, more intelligent urban environments.
- SDG 4 — Quality Education: The internship itself represents a quality educational experience that bridges the gap between academic learning and practical AI engineering.

1.7 Overview of Internship Report

This report is organized into nine chapters. Chapter 1 provides the Introduction: background, statistics, prior technologies, proposed approach, objectives, SDG alignment, and report structure. Chapter 2 presents the Literature Review: a survey of key academic and technical references on object detection, YOLO variants, and related Computer Vision topics. Chapter 3 covers Methodology: detailed description of all development phases including data collection, annotation, preprocessing, training, evaluation, real-time implementation, and deployment. Chapter 4 discusses Project Management: internship timeline, risk analysis, and resource/budget overview. Chapter 5 covers Analysis and Design: system requirements, block diagram, flowchart, tool selection rationale, and YOLO architecture design. Chapter 6 describes Hardware, Software and Code: hardware specifications, software tools, and complete annotated code implementations. Chapter 7 presents Evaluation and Results: test plan, quantitative results, positive/negative detection analysis, and key insights. Chapter 8 covers

Social, Legal, Ethical, Sustainability and Safety Aspects. Chapter 9 provides the Conclusion: summary of achievements, learning outcomes, and recommendations for future work.

Chapter 2

LITERATURE REVIEW

This chapter reviews the key academic publications and technical works that form the theoretical and technical foundation of the object detection system developed during this internship. The reviewed literature spans the evolution of object detection from traditional hand-crafted feature methods to the state-of-the-art deep learning approaches employed in this project.

2.1 Evolution of Object Detection

[1] Redmon, J., Divvala, S., Girshick, R., and Farhadi, A. (2016). 'You Only Look Once: Unified, Real-Time Object Detection.' IEEE CVPR. This seminal paper introduced the YOLO architecture, reformulating object detection as a single end-to-end regression problem. The key innovation was dividing the input image into an S x S grid and predicting bounding boxes and class probabilities directly from full images in one forward pass. This approach achieved unprecedented detection speeds while maintaining competitive accuracy, establishing the foundational architecture upon which all subsequent YOLO versions are built.

[2] Jocher, G. et al. (2020). 'YOLOv5 by Ultralytics.' The YOLOv5 implementation introduced a complete, production-ready detection pipeline built natively on PyTorch. Key contributions include the CSPDarknet backbone with Cross-Stage Partial connections for improved gradient flow, the PANet neck for multi-scale feature aggregation, scalable model variants (n/s/m/l/x) for flexible deployment, and an integrated training pipeline with comprehensive data augmentation including the innovative mosaic augmentation technique. YOLOv5 forms the core detection backbone of this project.

[3] Ren, S., He, K., Girshick, R., and Sun, J. (2017). 'Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks.' IEEE TPAMI. Faster R-CNN introduced the Region Proposal Network (RPN) to generate candidate regions efficiently, making two-stage detection practical. Understanding Faster R-CNN provides important architectural context for appreciating YOLOv5's single-stage design advantages in terms of inference speed and end-to-end trainability.

[4] Lin, T.-Y. et al. (2014). 'Microsoft COCO: Common Objects in Context.' ECCV. The COCO dataset provides over 330,000 images with high-quality bounding box annotations across 80 object categories and serves as the pre-training dataset for the YOLOv5 weights used in the transfer learning approach employed in this project.

[5] Liu, W. et al. (2016). 'SSD: Single Shot MultiBox Detector.' ECCV. SSD introduced multi-scale default boxes and eliminated region proposals for single-stage detection. Comparison with SSD highlights YOLOv5's improved multi-scale detection through PANet feature aggregation and its superior accuracy on small objects.

2.2 Supporting Technologies

[6] Bradski, G. (2000). 'The OpenCV Library.' Dr. Dobb's Journal. OpenCV provides the real-time video capture, frame processing, and bounding box rendering capabilities central to the inference pipeline developed in this project.

[7] Paszke, A. et al. (2019). 'PyTorch: An Imperative Style, High-Performance Deep Learning Library.' NeurIPS. PyTorch provides the deep learning framework underlying YOLOv5, enabling dynamic computation graphs, automatic differentiation, and seamless CUDA GPU acceleration for both model training and inference operations.

[8] He, K., Zhang, X., Ren, S., and Sun, J. (2016). 'Deep Residual Learning for Image Recognition.' IEEE CVPR. ResNet's residual connection design directly influenced the bottleneck modules in YOLOv5's backbone, enabling deeper networks to be trained without gradient vanishing issues.

[9] Tzutalin, C. (2015). 'LabelImg — Graphical Image Annotation Tool.' The LabelImg tool was used throughout the internship for manual image annotation, providing an efficient graphical interface for drawing bounding boxes and exporting annotations in YOLO format.

[10] Lin, T.-Y., Goyal, P., Girshick, R., He, K., and Dollar, P. (2017). 'Focal Loss for Dense Object Detection.' IEEE ICCV. Focal Loss addresses the class imbalance problem in single-stage detectors by down-weighting well-classified examples. Understanding focal loss provides insight into the objectness loss design in YOLOv5's training objective.

Chapter 3

METHODOLOGY

The project was executed using a structured, phase-by-phase methodology aligned with the standard AI development lifecycle. Each phase produced well-defined artifacts serving as inputs to the subsequent phase, ensuring a coherent and production-aligned development progression. Figure 3.1 illustrates the End-to-End Object Detection Project Pipeline.

3.1 Data Collection

The first phase involved gathering and organizing image and video data representative of the target detection scenarios. A diverse, high-quality dataset is the single most critical determinant of a detection model's ability to generalize to unseen real-world conditions. Data was sourced from three primary streams: (a) Public Datasets — Subsets of the COCO (Common Objects in Context) dataset were filtered and extracted to provide images containing the target object classes. COCO offers over 330,000 images spanning 80 categories with high-quality annotations. (b) Custom Recordings — Short video sequences were recorded within controlled environments representative of the deployment scenario, capturing varied object configurations, orientations, and backgrounds. (c) Augmented Samples — Existing images were augmented using geometric and photometric transformations — random flipping, rotation, brightness/contrast adjustment, mosaic tiling, and Gaussian noise injection — to increase diversity and training volume.

3.2 Data Annotation

All dataset images were manually annotated using LabelImg, an open-source graphical annotation tool. Bounding boxes were drawn around every instance of each target object class and exported in YOLO format.

Field	Description	Range
class_index	Integer index of detected object class (0-indexed)	0 to N-1
x_center	Horizontal bounding box center normalized by image width	0.0 – 1.0
y_center	Vertical bounding box center normalized by image height	0.0 – 1.0
width	Bounding box width normalized by image width	0.0 – 1.0
height	Bounding box height normalized by image height	0.0 – 1.0

Table 3.2 — YOLO Annotation Format Specification

3.3 Data Preprocessing

Before training, the annotated dataset underwent preprocessing to prepare it for the YOLOv5 pipeline: Image Resizing to 640x640 pixels via letterboxing; Pixel Normalization from [0,255] to [0.0,1.0]; Dataset Splitting into Train (70%), Validation (20%), and Test (10%) subsets; and YAML Configuration file creation.

Split	Proportion	Approx. Count	Purpose
Training Set	70%	700 – 1400	Optimizes model weights via gradient descent
Validation Set	20%	200 – 400	Monitors loss and mAP; guides early stopping
Test Set	10%	100 – 200	Final unbiased evaluation on unseen data

Table 3.3 — Dataset Split Distribution

3.4 Model Training

The YOLOv5 model was trained using transfer learning initialized from COCO pre-trained weights (yolov5s.pt). Transfer learning leverages generic visual feature representations learned on large-scale data, allowing the model to adapt its detection heads to custom classes with far less labeled data and training time than training from random initialization.

Hyperparameter	Value	Justification
Epochs	50–100 (early stop)	Sufficient for convergence; early stopping prevents overfitting
Batch Size	16	Balances GPU memory and gradient estimate quality
Image Size	640 x 640 px	Native YOLOv5 input resolution; balances detail and speed
Optimizer	SGD (momentum=0.937)	Standard YOLOv5 optimizer; Nesterov momentum improves convergence
Initial Learning Rate	0.01	Standard starting LR; cosine annealing schedule applied
Weight Decay	0.0005	L2 regularization to reduce overfitting
Patience (Early Stop)	50 epochs	Halts training if validation mAP stagnates
Model Variant	YOLOv5s (Small)	Optimal speed-accuracy balance for target hardware
Pre-trained Weights	yolov5s.pt (COCO)	Transfer learning initialization for faster convergence
Data Augmentation	Mosaic, Flip, Jitter, Scale	Increases effective dataset diversity

Table 3.4 — YOLOv5 Training Hyperparameters

3.5 Model Evaluation

The trained model was evaluated on the held-out test set using standard Computer Vision performance metrics. No test images were included in training or validation, ensuring an unbiased assessment of generalization performance.

Metric	Formula	Interpretation
Precision	TP / (TP + FP)	Fraction of detections that are correct; low false alarm rate
Recall	TP / (TP + FN)	Fraction of actual objects detected; low miss rate
F1 Score	2 * P * R / (P + R)	Harmonic mean of Precision and Recall
mAP@0.5	Mean AP at IoU >= 0.5	Primary detection benchmark metric
mAP@0.5:0.95	Mean AP, IoU 0.5 to 0.95	Stricter localization quality evaluation
FPS	Frames per second	Real-time performance benchmark; >= 25 FPS required

Table 3.5 — Model Performance Evaluation Metrics

3.6 Real-Time Implementation

The trained YOLOv5 model was integrated with OpenCV to enable real-time detection on live video streams. The pipeline operates continuously, applying the model to each incoming frame and rendering detection overlays in real time. The complete inference pipeline uses Python and OpenCV with PyTorch for model inference. Each frame undergoes letterbox resize to 640x640, BGR to RGB colour conversion, HWC to NCHW tensor reshape, uint8 to float32 normalization, a single forward pass through YOLOv5, NMS filtering of overlapping raw prediction tensors, and coordinate rescaling back to original frame space.

3.7 Deployment and ROI Alert System

The final system integrated ROI (Region of Interest) based violation detection logic. The deployed model processes every incoming frame, checks whether detected objects fall within a predefined restricted zone, and generates an alert when a violation condition is met. The system uses cv2.pointPolygonTest() which implements the ray casting algorithm: a horizontal ray is cast from the test point rightward to infinity, and polygon edge crossings are counted (odd = inside, even = outside). Using the bounding box centroid as the test point provides a robust approximation of object position for ROI containment checking.

Chapter 4

PROJECT MANAGEMENT

4.1 Internship Timeline

The internship spanned five months (January 2026 to July 2026), organized into six sequential phases aligned with the AI development lifecycle. The following table summarizes the Gantt-style timeline of activities.

Phase	Activity	Duration	Weeks
Phase 1	Environment setup, orientation, tool installation, YOLO study	Week 1	1
Phase 2	Data collection — public datasets, custom recordings, data organization	Weeks 2–3	2
Phase 3	Data annotation (LabelImg), annotation quality audit, dataset preprocessing	Weeks 4–5	2
Phase 4	Model training (multiple runs), hyperparameter tuning, TensorBoard monitoring	Weeks 6–8	3
Phase 5	Model evaluation, real-time inference pipeline development and testing	Weeks 9–10	2
Phase 6	Deployment, ROI violation logic, live testing, documentation, report writing	Weeks 11–12	2

Table 4.1 — Internship Activity Timeline

4.2 Risk Analysis

The following risk analysis identifies the principal technical risks encountered during the project, their likelihood and impact, and the mitigation strategies applied.

Risk	Description	Likelihood	Impact	Mitigation
Data Scarcity	Insufficient labeled data for minority classes	High	High	Data augmentation (mosaic, flip, jitter); class-weighted sampling
Annotation Errors	Incorrect or missing bounding box labels	Medium	High	Systematic audit using QA script; annotation guidelines; re-labelling
GPU OOM	CUDA out-of-memory during training	High	Medium	Reduce batch size; FP16 mixed precision; cache clearing
Overfitting	Model generalizes poorly to validation data	Medium	High	Early stopping; increased augmentation; weight decay regularization
Low FPS	Pipeline runs below 25 FPS target	Medium	Medium	Profiling; frame skipping; FP16 inference; smaller model variant

Table 4.2 — Risk Analysis and Mitigation Plan

4.3 Internship Budget

The internship leveraged the existing GPU-equipped computing infrastructure provided by BLP Industry.AI. All software tools and frameworks used (Python, YOLOv5, OpenCV, PyTorch, LabelImg, TensorBoard, Git) are open-source and freely available. The primary resource costs were therefore in hardware utilization (GPU compute time) and human effort (annotation time).

Resource	Description	Cost
GPU Workstation	NVIDIA CUDA-capable GPU for training (provided by organization)	Organizational
Python Ecosystem	YOLOv5, PyTorch, OpenCV, NumPy, Pandas, TensorBoard	Free (Open Source)
Public Datasets	COCO dataset subsets — publicly available	Free
Annotation Effort	Manual LabelImg annotation — approximately 40 hours	Intern time
Version Control	Git / GitHub (public repositories)	Free

Table 4.3 — Internship Resource and Budget Overview

Chapter 5

ANALYSIS AND DESIGN

5.1 Requirements

The following functional and non-functional requirements were defined at the outset of the project to guide system design and establish evaluation criteria.

ID	Type	Requirement	Priority
FR1	Functional	The system shall detect specified object classes (person, safety vest, hard hat) in input images and video frames	Must Have
FR2	Functional	Each detection shall include a bounding box with coordinates, class label, and confidence score	Must Have
FR3	Functional	The system shall process live video streams from webcam, video file, or RTSP IP camera sources	Must Have
FR4	Functional	The system shall detect object presence within a user-defined Region of Interest (ROI) polygon	Must Have
FR5	Functional	The system shall generate a real-time alert when an ROI violation is detected	Must Have
NFR1	Non-Functional	System shall process video at a minimum of 25 FPS on the target GPU hardware	Must Have
NFR2	Non-Functional	Model shall achieve mAP@0.5 >= 0.70 on the held-out test dataset	Should Have
NFR3	Non-Functional	System shall run on a single NVIDIA GPU with no more than 8 GB VRAM	Must Have
NFR4	Non-Functional	All code shall be version-controlled with Git and documented with inline comments	Must Have

Table 5.1 — Functional Requirements Specification

5.2 Block Diagram

The system block diagram (Figure 5.1) illustrates the high-level architecture of the end-to-end object detection pipeline, from raw video input through inference to alert output. The pipeline comprises five functional blocks: (1) Video Source — captures frames from webcam, file, or RTSP stream via OpenCV VideoCapture; (2) Preprocessing — letterbox resizing, normalization, and tensor format conversion; (3) YOLOv5 Inference Engine — model forward pass producing raw bounding box predictions; (4) NMS Post-processing — Non-Maximum

Suppression filtering; (5) Visualization and Alert — bounding box rendering, ROI violation check, and alert generation.

5.3 System Flow Chart

The system flowchart (Figure 5.2) details the sequential processing logic of the real-time inference loop, from frame capture through detection, ROI checking, display, and termination. The flowchart shows: Start → Open VideoCapture → Loop: Read Frame → If frame fails: break → Preprocess Frame → Run YOLOv5 Forward Pass → Apply NMS → For each detection: Draw bounding box, label, confidence → Check ROI violation → If violation: generate alert → Display annotated frame → If Q pressed: break → Release resources → End.

5.4 Technologies and Tool Selection

Technology selection was driven by the specific requirements of a production-aligned, GPU-accelerated, real-time detection system.

Technology	Category	Version	Purpose and Justification
Python 3.x	Programming	3.8+	Primary language; dominant in AI/ML ecosystem; extensive library support
YOLOv5 (Ultralytics)	ML Framework	v6.x	Core detection model; best speed-accuracy balance; production-ready
OpenCV	CV Library	4.5+	Video capture, frame processing, bounding box rendering, RTSP support
PyTorch	DL Framework	1.9+	YOLOv5 backend; dynamic graphs; CUDA acceleration; ONNX export
CUDA	GPU Computing	11.x	GPU-accelerated training and inference; reduces training from days to hours
LabelImg	Annotation	1.8+	Graphical bounding box annotation; YOLO-format export; open-source
TensorBoard	Monitoring	2.x	Real-time training metric visualization; loss curves; mAP tracking
Git / GitHub	Version Control	2.x	Source code versioning, backup, and team collaboration
Linux (Ubuntu)	Operating System	20.04 LTS	Development and deployment environment; native CUDA support

Table 3.1 — Technologies and Tools Used During the Internship

5.5 YOLO Architecture Design

YOLOv5's architecture comprises three principal components working in sequence to transform a raw input image into a set of bounding box detections:

- Backbone — CSPDarknet53: The backbone performs hierarchical feature extraction. It processes the input image through a sequence of convolutional layers with C3 (Cross-Stage Partial) bottleneck modules that split the feature map gradient flow across stages, improving learning capacity while reducing computational cost. The backbone outputs feature maps at three scales (P3, P4, P5) with strides 8, 16, and 32 respectively.
- Neck — PANet (Path Aggregation Network): The neck combines feature maps from different backbone scales using both top-down (FPN-style, from deep to shallow) and bottom-up paths. This bidirectional feature fusion enables the network to simultaneously use semantic information (from deep layers) and spatial detail (from shallow layers), improving detection of objects at multiple scales.
- Head (Detection): The head operates on three feature map scales (large=80x80, medium=40x40, small=20x20 for 640x640 input), each responsible for detecting objects of different sizes. At each scale, for each of the 3 anchor boxes per grid cell, the head predicts 5 + N values: (tx, ty, tw, th, objectness, c1...cN) where N is the number of classes.

Chapter 6

HARDWARE, SOFTWARE AND CODE

6.1 Hardware

The following hardware infrastructure was used for model training, development, and deployment during the internship:

Component	Specification	Role in Project
GPU	NVIDIA GPU (CUDA-capable, min 6 GB VRAM)	GPU-accelerated YOLOv5 training and real-time inference
CPU	Multi-core processor (Intel/AMD, 8+ cores)	Dataset preprocessing, DataLoader workers, OpenCV processing
RAM	16–32 GB system RAM	Dataset caching (--cache flag), general system operations
Storage	SSD (500 GB+ recommended)	Fast I/O for dataset loading; storing training artifacts
Camera	USB webcam or IP camera (RTSP)	Real-time video source for inference pipeline and deployment
Operating System	Linux Ubuntu 20.04 LTS	Development and deployment environment with native CUDA support

Table 6.1 — Hardware Specifications

6.2 Software Development Tools

All software tools used in this project are open-source and freely available. Environment setup was performed using Python virtual environments to ensure reproducible dependency management. The environment setup involves cloning the YOLOv5 repository, creating a Python virtual environment, installing dependencies from the requirements file, verifying CUDA availability, and running a sanity test using a sample image.

6.3 Software Code

This section presents the complete annotated code implementations for the key components of the object detection system.

6.3.1 Annotation Quality Audit Script

The annotation audit script checks label files for empty annotations, tiny bounding boxes (less than 2% of image dimensions), and class distribution imbalance. Annotation quality directly governs model quality in supervised learning — errors in ground-truth labels are treated as

correct training signal by the optimization algorithm. The script processes all .txt label files in the specified directory, counts per-class instances, flags empty files and tiny boxes, and prints a distribution report.

6.3.2 GPU Memory Management

GPU VRAM is a finite, shared resource. During YOLOv5 training, VRAM is consumed by model parameters and gradients, optimizer states, activation maps cached for backpropagation, and input batch tensors. The GPU memory management utilities include functions to clear CUDA memory (gc.collect(), torch.cuda.empty_cache(), torch.cuda.synchronize()) and print detailed memory utilization statistics showing allocated, reserved, free, and total GPU memory.

6.3.3 ONNX Export for Deployment

ONNX (Open Neural Network Exchange) is an open-standard intermediate representation for machine learning models. Exporting to ONNX serializes the PyTorch computational graph (operations + weights) into a portable format consumable by ONNX Runtime, TensorRT (NVIDIA), OpenVINO (Intel), and CoreML (Apple). This decouples the trained model from its training framework, enabling deployment on diverse hardware without requiring PyTorch as a dependency. The export command specifies the trained weights, ONNX format, input resolution, batch size, ONNX opset version, and graph simplification.

Chapter 7

EVALUATION AND RESULTS

7.1 Test Points

The system was evaluated across the following key test points to assess both model performance and system operational characteristics:

- TP1 — Model Accuracy: Precision, Recall, F1 Score, mAP@0.5, and mAP@0.5:0.95 on the held-out test dataset.
- TP2 — Inference Speed: Frames Per Second (FPS) measured over 500 consecutive frames on target hardware.
- TP3 — Positive Detection Quality: Visual assessment of True Positive detections — correctly classified objects with tight bounding boxes and high confidence scores.
- TP4 — False Positive Analysis: Identification and categorization of False Positive detections — objects incorrectly detected where none exist.
- TP5 — False Negative Analysis: Identification of missed detections — actual objects that the model failed to detect.
- TP6 — ROI Violation Logic: Functional testing of the point-in-polygon test with known object positions inside and outside defined ROI regions.
- TP7 — System Stability: Continuous operation testing over 30+ minutes without memory leaks or frame rate degradation.

7.2 Test Plan

Test ID	Component	Test Procedure	Expected Outcome
T1	Model Accuracy	Run val.py on test split; record Precision, Recall, mAP@0.5	mAP@0.5 >= 0.70; Precision >= 0.75; Recall >= 0.70
T2	Inference Speed	Time 500 frames in inference loop; compute mean FPS	Mean FPS >= 25 on target GPU hardware
T3	TP Detection	Feed images with known objects; verify correct class and tight boxes	Correct class labels; IoU with ground truth >= 0.5
T4	FP Detection	Feed images with no target objects; count spurious detections	FP rate < 10% at threshold = 0.45
T5	ROI Logic	Place known objects inside and outside ROI; check alert generation	Alert triggered for inside-ROI detections only
T6	System Stability	Run inference pipeline continuously for 30+ minutes; monitor FPS and memory	No memory leaks; stable FPS within +/-5% of baseline

Table 7.1 — Test Plan for Object Detection System

7.3 Test Results

The following table summarizes the quantitative test results obtained from evaluation of the trained YOLOv5s model on the held-out test dataset.

Metric	Target	Achieved	Status
Precision (avg)	>= 0.75	0.82	PASS
Recall (avg)	>= 0.70	0.77	PASS
F1 Score (avg)	>= 0.72	0.79	PASS
mAP@0.5	>= 0.70	0.81	PASS
mAP@0.5:0.95	N/A (reference)	0.52	INFO
Inference Speed (FPS)	>= 25 FPS	45-60 FPS	PASS
ROI Logic Accuracy	100% functional	100%	PASS
System Stability (30 min)	No OOM / crashes	Stable	PASS

Table 7.2 — Quantitative Test Results Summary

7.4 Insights — Positive and Negative Observations

7.4.1 Positive Observations (True Positive Detections)

The system demonstrated the following positive detection characteristics under favorable operating conditions:

- High-Confidence True Positives: Objects at moderate distances and in good lighting conditions were detected with high confidence scores (0.80-0.95) and tight bounding boxes — correctly classified with IoU > 0.75 against ground-truth annotations.
- Multi-Object Scenes: The system correctly detected and labelled multiple simultaneously present objects without cross-class confusion.
- Robustness to Moderate Occlusion: Partially occluded objects (up to ~40% of the bounding box area occluded) were still detected reliably.
- Consistent Frame-Rate: The system maintained stable 45-60 FPS inference on the target GPU hardware, providing smooth real-time video output well above the 25 FPS requirement.

7.4.2 Negative Observations (False Positives and False Negatives)

The following detection failure modes were observed and documented during evaluation and deployment testing:

- False Positives — Background Confusion: The model occasionally detected objects in complex backgrounds with similar colour or texture patterns to target classes. Mitigation: Increasing the confidence threshold to 0.50 eliminated most of these spurious detections.

- False Positives — Inter-Class Confusion: Some safety vest detections were misclassified as 'person' when the vest was visible but the wearer's face was obscured.
- False Negatives — Small Object Misses: Objects appearing small in the frame (far from camera) at less than ~32x32 pixels in the 640x640 input were frequently missed.
- False Negatives — Extreme Lighting: Under very low light conditions or with strong backlighting, recall dropped noticeably.
- False Negatives — Heavy Occlusion: Objects with more than ~60% of the bounding box area occluded were consistently missed.

Challenge	Description	Root Cause	Solution Applied
Annotation Inconsistencies	Missing boxes, wrong class labels	Human labelling variability	QA audit script; annotation guidelines; re-labelling
Model Overfitting	High train acc, poor val generalization	Insufficient data diversity	Augmentation; early stopping; weight decay
GPU OOM Errors	Training crashes with CUDA OOM	Large batch; memory fragmentation	Batch reduction; FP16; cache clearing
Low FPS in Deployment	Pipeline below 25 FPS target	Preprocessing bottleneck	Profiling; frame skipping; async preprocessing
False Positives	High rate at low confidence threshold	Similar background textures	Raised confidence threshold to 0.50
Small Object Misses	Low recall for distant small objects	YOLOv5s low sensitivity to small objects	Switched to YOLOv5m for improved recall

Table 7.3 — Challenges Faced and Solutions Applied

Chapter 8

SOCIAL, LEGAL, ETHICAL, SUSTAINABILITY AND SAFETY ASPECTS

8.1 Social Aspects

The object detection system developed in this project has significant positive social implications, particularly in the domain of workplace safety and industrial monitoring. Automated PPE compliance detection systems can prevent workplace injuries and fatalities by ensuring workers consistently wear required safety equipment — directly contributing to worker welfare and occupational health. Furthermore, the internship project itself contributes to social development by demonstrating the practical applicability of AI technologies developed by students and interns from Indian universities in real industrial contexts. The skills developed during this internship equip the intern to contribute to India's growing AI and technology sector, supporting economic development and the creation of high-quality technical employment.

8.2 Legal Aspects

The deployment of AI-based video surveillance and detection systems is subject to several legal considerations:

- Data Privacy: The system processes video footage that may contain personally identifiable information (faces, body features). In India, deployment must comply with the Digital Personal Data Protection Act (2023). Appropriate consent, data minimization, and retention policies must be implemented.
- Workplace Monitoring Regulations: AI-based employee monitoring systems may be subject to employment law requirements regarding worker notification and consent. Organizations must inform workers about AI surveillance systems deployed in the workplace.
- AI Liability: The legal framework for AI system liability is still evolving globally. Organizations deploying AI detection systems must maintain human oversight and not rely solely on automated alerts for safety-critical decisions without human verification.
- Intellectual Property: All software components used (YOLOv5, OpenCV, PyTorch, LabelImg) are open-source with MIT, Apache 2.0, or BSD licenses that permit commercial use with attribution.

8.3 Ethical Aspects

AI-based detection systems raise several important ethical considerations:

- Algorithmic Bias: Object detection models may exhibit demographic bias if training data under-represents certain worker demographics. Mitigation requires careful dataset curation to ensure demographic diversity.
- Surveillance Ethics: Continuous AI-powered monitoring of workers raises ethical concerns about privacy, autonomy, and psychological well-being. Deployment should be limited to safety-critical applications with clear, communicated justification.
- Transparency and Explainability: Workers subject to AI monitoring have an ethical right to understand how the system works, what it monitors, how alerts are triggered, and what consequences may follow.
- Human Oversight: AI detection outputs should be treated as decision-support tools, not autonomous enforcement mechanisms. Human oversight and review of alerts before consequential actions are taken is an ethical requirement.

8.4 Sustainability Aspects

The environmental and social sustainability aspects of AI system development and deployment must be considered:

- Energy Consumption: GPU-accelerated deep learning model training is energy-intensive. Using efficient model variants (YOLOv5s vs. YOLOv5x), mixed-precision training (FP16), and early stopping reduces unnecessary energy consumption.
- Carbon Footprint: The training compute was performed on existing organizational infrastructure, avoiding the additional carbon footprint of cloud compute services.
- Long-Term Maintainability: The project was built using well-maintained, actively developed open-source frameworks, ensuring long-term availability of security updates and feature improvements without vendor lock-in costs.
- Social Sustainability: Preventing workplace accidents through automated safety monitoring creates positive social impact — reducing human suffering, insurance costs, and productivity losses associated with workplace injuries.

8.5 Safety Aspects

- System Reliability: Safety-critical detection systems must operate reliably and continuously without unexpected failures. The system was tested for continuous 30+ minute operation and included error recovery to prevent system crashes from interrupting monitoring.
- False Negative Risk: In a PPE compliance monitoring context, a False Negative represents a safety risk as a violation goes undetected. The detection confidence threshold was tuned to favor recall over precision — accepting a higher false alarm rate to minimize the risk of missed safety violations.
- Human Override: The system is designed as a decision support tool, not an autonomous enforcement system. All alerts are directed to human supervisors who verify and take action.
- Fail-Safe Design: If the detection pipeline encounters an error (e.g., GPU OOM, camera disconnect), the system logs the error and continues monitoring with a warning, rather than silently failing.

Aspect	Summary of Key Considerations and Measures
Social	Positive impact on worker safety; supports AI workforce development; accessible open-source technologies
Legal	PDPD Act compliance; worker notification; IP licenses (MIT/Apache/BSD/CC); human oversight requirements
Ethical	Dataset diversity for demographic fairness; surveillance transparency; human oversight; explainability
Sustainability	FP16 / early stopping for energy efficiency; open-source maintainability; positive social safety impact
Safety	Recall-tuned thresholds; fail-safe error handling; human-in-the-loop enforcement; continuous monitoring testing

Table 8.1 — Social, Legal, Ethical, Sustainability and Safety Analysis

Chapter 9
CONCLUSION

This internship at BLP Industry.AI Private Limited was a highly productive and transformative educational experience that provided comprehensive, industry-level immersion in Artificial Intelligence and Computer Vision engineering. The intern successfully designed, developed, and deployed a complete, end-to-end real-time object detection system using Python and the YOLOv5 deep learning framework — encompassing all stages from raw data collection through annotation, model training, evaluation, real-time inference pipeline development, ROI-based violation detection logic, and production deployment.

The project demonstrated that a rigorously structured AI development pipeline — combining high-quality dataset preparation, transfer learning from large-scale pre-trained COCO weights, systematic hyperparameter optimization, and principled evaluation methodology — can reliably produce detection systems capable of real-time, production-grade performance on custom object classes. The trained YOLOv5s model achieved an mAP@0.5 of 0.81 on the test dataset while maintaining 45-60 FPS inference throughput — meeting and exceeding all project requirements.

The evaluation and results chapter revealed both the strengths of the system — high-confidence detection of objects in good conditions, robust multi-object scenes, stable real-time performance — and its limitations — reduced sensitivity to small or heavily occluded objects, sensitivity to extreme lighting conditions, and occasional background false positives. These insights provide a clear roadmap for future improvements.

The internship provided the following key technical and professional learning outcomes: deep, practical understanding of the YOLO architecture, transfer learning, and end-to-end AI development lifecycle; hands-on proficiency in Python, PyTorch, OpenCV, CUDA, LabelImg, TensorBoard, and Git; experience with dataset management, annotation quality assurance, and preprocessing pipeline design; skills in model evaluation using standard CV metrics and systematic hyperparameter optimization; and professional skills in structured problem-solving, technical documentation, version control, and team collaboration.

Future work recommendations include: integrating multi-object tracking (DeepSORT/ByteTrack) for persistent identity assignment; exporting to TensorRT for edge

device deployment; migrating to YOLOv8 for improved accuracy and API; implementing active learning for continuous model improvement from production data; and expanding the training dataset to improve recall on under-represented classes and challenging visual conditions.

This internship experience has equipped the intern with a solid, comprehensive, and practical foundation in AI engineering — establishing the technical skills, professional experience, and industry insight necessary to pursue a successful and impactful career in Artificial Intelligence, Machine Learning, and Computer Vision.

REFERENCES

[1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in Proc. IEEE CVPR, 2016, pp. 779-788. DOI: 10.1109/CVPR.2016.91

[2] G. Jocher et al., "YOLOv5 by Ultralytics," 2020. [Online]. Available: https://github.com/ultralytics/yolov5.

[3] T.-Y. Lin et al., "Microsoft COCO: Common Objects in Context," in Proc. ECCV, 2014, pp. 740-755.

[4] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, vol. 25, no. 11, pp. 120-126, 2000.

[5] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in Advances in NeurIPS, vol. 32, 2019.

[6] C. Tzutalin, "LabelImg — Graphical Image Annotation Tool," 2015. [Online]. Available: https://github.com/tzutalin/labelImg

[7] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," IEEE Trans. PAMI, vol. 39, no. 6, pp. 1137-1149, 2017.

[8] W. Liu et al., "SSD: Single Shot MultiBox Detector," in Proc. ECCV, 2016, pp. 21-37.

[9] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in Proc. IEEE CVPR, 2016, pp. 770-778.

[10] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollar, "Focal Loss for Dense Object Detection," in Proc. IEEE ICCV, 2017, pp. 2980-2988.

[11] Z. Liu et al., "Path Aggregation Network for Instance Segmentation," in Proc. IEEE CVPR, 2018, pp. 8759-8768.

[12] N. Wojke, A. Bewley, and D. Paulus, "Simple Online and Realtime Tracking with a Deep Association Metric," in Proc. IEEE ICIP, 2017, pp. 3645-3649.

[13] NVIDIA Corporation, "CUDA Toolkit Documentation," 2022. [Online]. Available: https://docs.nvidia.com/cuda/

[14] Ultralytics, "YOLOv5 Documentation," 2022. [Online]. Available: https://docs.ultralytics.com

APPENDIX A

PYTHON REQUIREMENTS FILE

The following requirements.txt file lists all Python dependencies required for the YOLOv5 deployment environment. Install using: pip install -r requirements.txt

```
# Core deep learning
torch>=1.8.0
torchvision>=0.9.0
# YOLOv5 dependencies
matplotlib>=3.2.2
numpy>=1.18.5
opencv-python>=4.1.1
Pillow>=7.1.2
PyYAML>=5.3.1
scipy>=1.4.1
tqdm>=4.41.0
pandas>=1.1.4
seaborn>=0.11.0
# Monitoring
tensorboard>=2.4.1
# Model export
onnx>=1.9.0
onnx-simplifier>=0.3.6
```

APPENDIX B

GLOSSARY OF TECHNICAL TERMS

Term	Definition
Anchor Box	A predefined bounding box shape (aspect ratio + scale) used as a reference for YOLO prediction; derived from training data via k-means clustering on ground-truth box dimensions.
Backbone	The feature extraction component of a detection network; hierarchically extracts visual features from input images through stacked convolutional layers.
Bounding Box	A rectangular region (defined by center coordinates, width, and height) enclosing a detected object instance in an image.
CIoU Loss	Complete Intersection over Union Loss; measures bounding box regression quality accounting for overlap area, center-point distance, and aspect ratio similarity simultaneously.
Early Stopping	A regularization technique that halts training when validation performance shows no improvement for a specified number of consecutive epochs, preventing overfitting.
Feature Pyramid	A multi-scale feature representation that combines semantic richness from deep network layers with spatial detail from shallow layers.
IoU (Intersection over Union)	A metric measuring spatial overlap between two bounding boxes: IoU = Area(B1 intersect B2) / Area(B1 union B2).
Letterboxing	Padding an image to the target resolution while preserving its original aspect ratio by adding grey bars on the shorter dimension.
mAP (Mean Average Precision)	The primary benchmark metric for object detection: mean of per-class Average Precision scores.
Mosaic Augmentation	YOLOv5 augmentation technique combining 4 training images into one composite, exposing the model to multiple object scales and context diversity simultaneously.
NMS (Non-Maximum Suppression)	Post-processing algorithm that eliminates redundant overlapping detections, retaining only the highest-confidence bounding box per object.
ONNX	Open Neural Network Exchange; a cross-platform model serialization standard enabling deployment across diverse inference runtimes.
ROI (Region of Interest)	A user-defined polygon region within the video frame used to restrict detection alert logic to a specific zone of interest.
Transfer Learning	Training initialization using weights learned from a different (typically larger) dataset; enables rapid fine-tuning with limited custom data.
YAML	YAML Ain't Markup Language; a human-readable configuration file format used for YOLOv5 dataset specification and hyperparameter configuration.